

The Alberta Buck - Platform Architecture (DRAFT v0.1)

Perry Kundert

2026-07-03 12:00:00

Bootstrapping the BUCK monetary system requires three deliverables that share one foundation:

1. **High-resolution simulation:** 10^3 - 10^4 agents (citizens earning, living and saving; investors; businesses; governments) transacting against the real BUCK contracts and AMM pools over multi-year horizons.
2. **Live-interactive demonstrations:** browser-hosted simulations – in web sites and slide decks – where an audience member creates agents, mints BUCK_CREDIT, executes identity-signed transfers, and watches background agents rebalance real AMM pools under simulated time and price feeds.
3. **Production platforms:** JS, native, and Python libraries with which the initial identity registries, insurers, and credit unions integrate their back-office systems – signing Identities, generating BuckCredit NFTs, producing BUCK account products – and on which the first BUCK native wallets are built (Identity "fountain" management, Ethereum transaction construction and signing).

The shared foundation is (a) a backend-agnostic **ChainSession** API with expectation checking and journaling, spanning anvil, public testnets, Tenderly overlays, and in-process JS EVMs (Tevm); and (b) a Rust mathematical kernel – monetary arithmetic and the Identity/proving cryptography – compiled to WASM for JavaScript and bound natively for Python, so every platform predicts bit-identical fees, demurrage, depreciation, and proofs.

Contents

1	Decisions of record	3
2	Architecture: three layers	4
2.1	Layer 1: <code>ChainSession</code> – one testing API over every backend	4
2.2	Layer 2: the deterministic kernel – one Rust workspace, two bindings	5
2.3	Layer 3: agent platforms – Python today, JS as a peer	6
3	Repository layout and transition strategy	7
4	Phase plan	7
4.1	Phase 0: seam + scaffold [status: DONE 2026-07-03]	7
4.2	Phase 1: JS <code>ChainSession</code> + walking-skeleton agents [status: DONE 2026-07-03]	8
4.3	Phase 2: <code>buck-math</code> + bindings [status: DONE 2026-07-03] . .	8
4.4	Phase 3: <code>buck-identity</code> port [status: DONE 2026-07-03] . .	9
4.4.1	Design of record (banked 2026-07-03)	10
4.5	Phase 4: JS sim platform + live-interactive demos [status: IN PROGRESS]	11
4.5.1	Design of record (banked 2026-07-03)	11
4.5.2	The JS agent doctrine (banked 2026-07-04)	14
4.6	Phase 5: scale-up	15
4.7	Phase 6: partner SDKs + wallet productization	15
5	Risks and open questions	15

1 Decisions of record

Settled 2026-07-02/03; revisit deliberately, not accidentally.

- **Population target:** 10^3 - 10^4 agents. Some "fat" (one agent driving many identities), but BUCK_CREDIT-bearing agents must stand alone. This scale does NOT require abandoning anvil: `ChainSession` over anvil is the primary sim backend; `pyrevm` (in-process revm) remains an **optional** accelerator gated by identical-vector tests against anvil.
- **No pure-Rust simulator.** Python (and later JS) orchestrate; Rust holds only the deterministic math kernels. Understandability of the sim and its agents is paramount: the chain API stays tidy, agents stay simple.
- **JS live-interactive demos are a requirement**, not a nice-to-have. Record/replay does not cut it: demos must let someone set up simulated agents, create BuckCredit, watch balances adjust, and execute transfers using real identities while background agents rebalance the real AMM pools on simulated time and prices. This implies a JS agent-simulation platform that is a **peer** of `alberta_buck/sim` – justified because (a) most researchers will never run Python, (b) most integrating organizations prefer JS, and (c) the demo wallet should be the **real** JS wallet signing real transactions.
- **The full Identity and proving stack must be available to both JS and Python** for sim and application R&D.
- **Layout:** the existing Python `alberta_buck/` keeps working throughout. New Rust, JS, and Python implementations land under `core/`, and `alberta_buck/` transitions onto them incrementally. Dependency rule: `core` never imports `alberta_buck`; `alberta_buck` may import `buck_core`.
- **Tests:** every ported/new core capability is covered in Rust, Python and JS – minimal in Rust (the kernel is small and vector-driven), primarily in Python and JS (the platforms and bindings).
- **The Solidity contracts are the mathematical specification.** Golden vectors generated by forge from the deployed-code paths are the cross-language ground truth; Rust/Python/JS suites assert byte-equality against the same vector files.

2 Architecture: three layers

Layer 3	agent platforms (scenarios, TOML experiments, demos)	alberta_buck/sim (Python) experiment harness, sweeps, eqmetrics	core/js sim (TS/JS) live-interactive browser demos
Layer 1	ChainSession API (send/call/deploy, expect=OK/REVERT, JSONL journal)	Web3Session (Python) anvil testnet Tenderly [pyrevm] _____ shared JSONL journal _____/	viem/tevm session (JS) tevm-in-browser anvil testnet
Layer 2	deterministic kernel (Rust -> WASM + Py03)	core/rust: buck-math, buck-identity, buck-wallet golden vectors from forge = the spec	

Layer numbering is historical (session API first, kernel second, platforms third); the dependency order is kernel <- session <- platforms.

2.1 Layer 1: ChainSession – one testing API over every back-end

The API is the product; execution engines plug in underneath. It is the formalization of what `alberta_buck/sim/chain.py`'s `Chain` already was, plus two ideas from the bootstrap requirements:

- **Expectations:** every state-changing operation declares whether it is expected to succeed or revert (`expect=OK` default, `expect=REVERT`). An unexpected revert raises (as today); an unexpected success logs a warning and is recorded. Expectation mismatches are first-class results, visible in the journal and countable by tests.
- **Journal:** an append-only JSONL record of every operation – tag, op, function, sender, expectation, outcome, gas, tx hash, block, error. Journals make runs auditable, diffable across backends and languages, replayable for demos, and memoizable ("the test infrastructure remembers our results"). Journaling is off by default (zero hot-path cost) and enabled per-session or via `BUCK_JOURNAL=path`.

Journal entry schema (one JSON object per line; unknown fields ignored):

field	type	meaning
i	int	1-based sequence number within the session
tag	str	caller-supplied label ("deploy:Buck", "saver3:buy")
op	str	send / deploy (calls are not journaled)
fn	str	contract function name or "constructor"
sender	str	from-address
expect	str	ok / revert
outcome	str	ok / revert
matched	bool	outcome == expectation
gas	int	gasUsed from the receipt
tx	str	transaction hash (hex)
block	int	block number
err	str	revert reason when outcome is revert

Backends, fidelity-vs-speed:

backend	runs in	role
anvil (foundry)	local process	PRIMARY sim backend; fidelity anchor
public testnets	remote	staging, partner integration
Tenderly virtual nets	remote	shared demos, partner staging
Tevm (JS, ethereumjs)	browser / node	live-interactive demos, JS sim
pyrevm (optional)	in-process	acceleration, iff vectors identical

2.2 Layer 2: the deterministic kernel – one Rust workspace, two bindings

Following the proven `holo-mutualcredit` shape (core crate -> `mutualcredit-js` wasm-bindgen package -> Holochain zomes), modernized and extended with a Python binding:

- **buck-math**: integer-only monetary arithmetic – BUCK_CREDIT declining-balance depreciation (discrete whole-year compounding, linear partial year, 100-year cap), demurrage / buckSeconds carrying apportionment, `fundingFactor()`, PID controller arithmetic (rescaled ppm), fees. No floats anywhere; U256/ppm/bp semantics matching Solidity. `no_std`-compatible core, preserving the Holochain zome option.
- **buck-identity**: BN254 (arkworks `ark-bn254`, replacing pure-Python `py_ecc`), Poseidon, Pointcheval-Sanders credentials, ElGamal, Schnorr, Chaum-Pedersen, the NIZK envelope/receipt/unilateral A1/A2 transcripts. This is simultaneously the wallet's production cryptography

and the sim's scaling bottleneck (identity registration is why `identity-cache.json` exists).

- **buck-wallet**: Identity "fountain" management, transaction construction and signing (alloy), account products. Later backed by a Holochain DHT – the same crates compile into zomes.
- **bindings/js**: wasm-bindgen + wasm-pack -> npm package (browser + Node).
- **bindings/py**: PyO3 + maturin -> wheels.

Bit-identical discipline: forge writes golden vectors (`test/vectors/math-*.json`) exercising the Solidity paths; Rust `cargo test`, Python `pytest`, and JS `node --test` all assert byte-equality against the same files. The existing Python `alberta_buck/wallet` remains the executable reference during the port, then becomes a thin shim over the PyO3 binding.

2.3 Layer 3: agent platforms – Python today, JS as a peer

Python (`alberta_buck/sim`) keeps its role: the research harness – TOML experiments, interventions, sweeps, eqmetrics, plots. It gains speed and rigor from Layers 1-2 but is not rewritten.

JS (`core/js`) grows into a peer platform in stages:

1. A **ChainSession** over viem with two transports: Tevm (in-process, browser-capable) and HTTP (anvil / testnets) – so JS agents can join the SAME anvil instance as a running Python sim (mixed-language sims), or run standalone in the browser.
2. A small, understandable agent API (mirroring the Python `setup()` / `act(day, tick)` shape) with simulated-time progression and price-feed plumbing, sharing the scenario TOML vocabulary.
3. The interactive demo layer: create agents from the page, mint Buck-Credit, transfer with REAL identities (the wasm `buck-identity` kernel proving in-browser), while whale/arb agents rebalance the AMM pools in the background.

Divergence between the two platforms is controlled by (a) both consuming the same compiled kernels for all money/identity math, (b) the shared journal format making cross-platform runs diffable, and (c) a canonical scenario whose Python-run and JS-run journals are compared in CI (Phase 4+).

3 Repository layout and transition strategy

core/	
README.md	layout, build/test commands, dependency rules
python/	buck_core package (pyproject; sessions, later Py03 re-exports)
buck_core/	session.py (Expect/Journal/Web3Session), artifacts.py
tests/	pytest suite (primary)
js/	@alberta-buck/core (ESM; journal now, session+agents next)
src/ test/	node --test suite (primary; zero deps until tevm lands)
rust/	cargo workspace: buck-math (+ buck-identity, buck-wallet, bindings/js, bindings/py as they land); minimal tests
vectors/	cross-language fixtures (journal-sample.jsonl, ...)
test/vectors/	forge-generated golden math/identity vectors (the spec)
alberta_buck/	UNCHANGED consumer; transitions module-by-module

Transition mechanics:

- Bootstrap import shim: `alberta_buck/__init__.py` prepends `core/python` to `sys.path` when running from a checkout, until `buck-core` ships as an installable wheel (Phase 2). `alberta_buck/sim/chain.py` keeps exporting `Chain` / `load_artifact` / `repo_root` forever; they become re-exports.
- Each phase flips specific `alberta_buck` internals onto `core` and states the regression gate that proves nothing moved (usually: routing smoke + canonical equilibrium vector unchanged).
- Nothing in `core` may import `alberta_buck`.

4 Phase plan

Each phase ships value standing alone, has explicit acceptance criteria, and is banked with vectors/journals before the next begins.

4.1 Phase 0: seam + scaffold [status: DONE 2026-07-03]

`core/` scaffold; Python `ChainSession` (expectation + journal) extracted from `sim/chain.py`, which becomes a delegating subclass; JS journal reader; Rust workspace seed; Make targets (`core-test{,-py,-js,-rust}`).

- *Accept:* routing smoke green on the new session; core Python/JS/Rust tests green; the same journal fixture parses identically in Python & JS.

- *Met*: routing smoke green; suites 9 py / 5 js / 3 rust; a journaled live sim run (491 ops, 153M gas, zero mismatches) summarized by the JS reader.

4.2 Phase 1: JS ChainSession + walking-skeleton agents [status: DONE 2026-07-03]

viem-based session with Tevm and anvil transports; journal writer emitting the shared schema; a whale + one arb agent in TS/JS.

- *Accept*: the JS agent pair runs (a) against a Python-deployed anvil sim (mixed-language) and (b) standalone on Tevm in Node; journals validate.
- *Met*: (a) `test_js_join_web3.py` – Python deploys the routing stack, the JS `PinWhale` snaps the real TOKEN/USDC pool along a +/- 5% path and the `RoundTripTrader` round-trips through it (incl. a DECLARED SPL revert), journal validated by the Python reader, final spot within 2% of target; (b) `onepool.tevm.test.js` – the same agents on a V3 pool built wholly in-process on Tevm (~2.7s). Tevm notes: pass `chain: null` + explicit gas with local accounts; keep deploy gas $\leq$ ~15M (block gas limit); decode revert reasons via `e.walk(ContractFunctionRevertedError)(reason or data.errorName)`.

4.3 Phase 2: buck-math + bindings [status: DONE 2026-07-03]

Integer kernel (depreciation, demurrage/buckSeconds, fundingFactor, PID ppm, fees); forge-generated golden vectors; wasm-pack npm package; PyO3 wheel; `alberta_buck` sim mirrors (e.g. the agents' fundingFactor) and wallet fee math flip to the binding.

- *Met*: `test/MathVectors.t.sol` drives the REAL code paths (`_depreciate`, public `feeOwing` incl. its `raw<=0` gate, `_carryingTransfer`, `Direct compute()` with `cache/dTMax/anti-windup`, `fundingFactor`) into 347 golden rows; the `buck-math` crate (no `_std`, dependency-free, hand-rolled 256-bit `mulDiv`, overflow-checks in every profile) reproduces all of them, as do the PyO3 binding (`buck_core.buck_math`, plain-cargo `cdylib`) and the wasm-bindgen package (`core/js/wasm`, `BigInt` API) – suites: cargo 16, pytest 15, node 12. NOTE: no exact-integer mirror existed in the Python sim to flip (the agents' funding factor is deliberately

BEHAVIORAL float math with stochastic knobs), so the "canonical vector unchanged" gate was vacuous; the kernel is the canonical prediction surface for partner SDKs, wallets, and future sim/eqmetrics uses. Builds: `make nix-core-build{-py,-wasm}`; the repo venv hosts `alberta_buck` AND -e `core/python` together.

- *Accept*: Rust/Python/JS byte-equal on the forge vectors; canonical equilibrium vector unchanged after the sim flips to the binding.

4.4 Phase 3: buck-identity port [status: DONE 2026-07-03]

arkworks BN254 + Poseidon + PS + ElGamal + Schnorr + Chaum-Pedersen + NIZK transcripts; PyO3 + wasm bindings; `alberta_buck/wallet` delegates.

Includes the PAPERS: the org design docs that demonstrate identity-related proofs with Python example code (the executable transcript docs – `alberta-buck-identity-example.org` and the notes/receipt/paper family rendered via `make nix-venv-doc-%` – plus any inline identity examples) are updated to exercise the NEW implementation through the `buck_core.buck_math`-style identity binding, so the published documents demonstrate the production kernel, not the `py_ecc` reference. The Python reference in `alberta_buck/wallet` remains during the port as the executable spec.

- *Accept*: existing wallet vectors (`alberta_buck/wallet/vectors.py`, SNARK fixtures) reproduce bit-identically; on-chain registration in the sim verifies; identity-cache regeneration matches; in-browser proof generation demonstrated; the executable identity papers render green against the new binding.
- *Met* (2026-07-03): `test/vectors/identity.json` regenerates BYTE-identically under the kernel backend and the full forge suite (497) stays green against it, incl. the SNARK-fixture-pinned `issuer_reenc` section; the routing smoke registers every EOA agent on-chain with real NIZKs, and a dedicated gate proves cold identity-cache regeneration BACKEND-INVARIANT end-to-end on anvil (`test_identity_cache_regen.py`; the committed cache itself is an append-only mix of code epochs, so cross-epoch byte-equality is explicitly not the invariant); the browser demo (`core/js/demo/identity-proofs.html`, `make nix-core-demo-identity`) generates and verifies the full registration ceremony client-side, headlessly smoke-tested over the exact web bundle; all four executable papers assert `backend() = "kernel"` in their transcripts and render green

(`alberta-buck-identity-example.org` gained its `exec-render` target and a seeded deterministic transcript; `alberta-buck-proofs.org` stays `py_ecc` by design). Cross-language ground truth: `core/vectors/identity-kernel-vectors.json` (nonce -inclusive, emitted by the `py` reference) replayed byte-for-byte by cargo (3 tests), `pytest` (30), and `node` (27) suites.

4.4.1 Design of record (banked 2026-07-03)

Kernel scope (`core/rust/buck-identity`): the deterministic crypto core – BN254 G1/G2 arithmetic and pairing checks (EVM `ecPairing` semantics: `pairing_check([(G1,G2),...]) -> bool`), keccak Fiat-Shamir transcripts (`keccak_scalar/=identity_scalar`), circomlib Poseidon (constants build-generated from a vendored byte-identical `poseidon_constants.json`), and every sigma protocol the wallet implements: PS sign/verify/rerandomize, ElGamal, registration NIZK, Chaum-Pedersen approve, issuer Schnorr batch binding, verifiable decryption, A2 issuer re-encryption binding, deposit coupling, B1 depositor binding, note commitments / nullifiers / id-hashes / `identity_leaf`. NOT kernel: canonical-JSON identity serialization, the AB-RCPT/1 envelope, receipt build/verify orchestration, rendering, the Merkle tree container (Python/JS keep those; the tree’s Poseidon calls hit the kernel), and Groth16 SNARK proving (stays `snarkjs/rapidsnark`).

Determinism rule: kernel functions take EVERY nonce/blind as an explicit argument – no rng inside the kernel. Randomness stays language-side, so the pinned Python `random.Random` streams (`vectors.py`, seeded sims, paper sessions) are preserved draw-for-draw and the emitted artifacts stay byte-identical.

Delegation architecture: each `alberta_buck/wallet` module keeps its exact public API and dataclasses; provers draw their nonces in Python (in today’s order, BEFORE branching), then either call the kernel (default when the binding is importable) or fall through to the retained `py_ecc` reference (forced via `BUCK_IDENTITY_BACKEND=py`). `bn254.add/mul/neg` gain a G1 fast path (`py_ecc` FQ tuple in, FQ tuple out); `pairing` stays `py_ecc` at the `bn254` level (protocol verifiers use kernel pairing checks wholesale). `poseidon.poseidon()` delegates the same way.

Dependency decision: unlike zero-dep `buck-math`, `buck-identity` takes `ark-bn254/=ark-ec/=ark-ff 0.5` (hand-rolling a correct optimal-ate pairing is not a good risk) plus `tiny-keccak`; `std` (not `no_std`) – `arkworks` is `wasm-clean`, and the `no_std` flip stays mechanical (`default-features = false`) if a Holochain zome ever needs it.

Binding surface: `PyO3` module `buck_core.buck_identity` (plain-cargo

build like `buck_math`; `num-bigint` feature so Python ints cross the boundary; points as `(x, y)` int tuples, G2 as coefficient-pair tuples, proofs as flat tuples the Python shims wrap back into dataclasses). wasm package `core/js/wasm/buck_identity` with 0x-hex strings at the ABI and a `core/js/src/identity.js` wrapper exposing the BigInt-native API (plus the canonical-JSON identity serializer for in-browser registration).

Vector gates (in addition to the acceptance list above):

1. Regenerating `test/vectors/identity.json` (`emit-vectors`), the golden receipt texts, and `identity-cache.json` under the kernel backend produces ZERO git diff; the full forge suite stays green.
2. A new nonce-inclusive cross-language fixture, `core/vectors/identity-kernel-vectors.json`, emitted by the `py_ecc` REFERENCE path (`BUCK_IDENTITY_BACKEND=py`), replayed bit-identically by the cargo, pytest, and node suites (every prove call with recorded nonces, verify + negative cases, poseidon/keccak/curve-op rows).

Papers plan: `alberta-buck-notes-flow.org`, `alberta-buck-paper.org`, `alberta-buck-receipt.org` call the wallet API, so kernel delegation flips their executed sessions automatically; each gains a visible production-kernel assertion in its transcript. `alberta-buck-identity-example.org` additionally gets an executable render target (today it renders committed RESULTS via the generic `doc-%`) and its raw `py_ecc` curve-pedagogy cells move to the binding's curve ops. `alberta-buck-proofs.org` (hand-rolled sigma pedagogy on raw `py_ecc`, no wallet imports) is explicitly out of scope.

4.5 Phase 4: JS sim platform + live-interactive demos [status: IN PROGRESS]

Agent API + simulated time + price feeds on Tevm-in-browser; the demo scenario (interactive agent creation, BuckCredit mint, identity-signed transfers, background AMM rebalancing); slide-deck/web embedding.

- *Accept:* the demo scenario runs wholly in-browser with real identities; a canonical mini-scenario's JS and Python journals agree.

4.5.1 Design of record (banked 2026-07-03)

Risk killed first (spike, now the Stage-0 test): Tevm's EVM ships the BN254 precompiles – `IdentityRegistry.register` verified a wasm-kernel-generated registration NIZK ON-CHAIN in-process in ~180 ms (a unicode

identity, at that), replay from another account reverted as expected, and `evm_setNextBlockTimestamp` + `evm_mine` give simulated time. No custom-compile shim needed: the full identity ceremony – kernel proving, real Solidity verification – runs wholly in JS. Gotcha: `BN254.sol` struct components are UPPERCASE `X/=Y` at the ABI (viem needs object keys to match exactly).

Stages (1-4 DONE 2026-07-03):

1. **Buckworld on Tevm** (node-gated): an artifacts bundle build step (`out/*.json` -> one importable module: nodefs stays node-only, the browser imports the bundle – `buildOnePool` already takes an injected `artifacts(name)`); a `buildBuckWorld` scenario builder deploying the REAL stack (`IdentityRegistry` + `BuckCredit` + `BuckKControllerDirect` + `Buck`, then `TOKEN/USDC V3` pools via `SimLP`) mirroring `deploy.py`'s order; identity onboarding helper (kernel ceremony -> register); simulated-time helpers (`day ticks` -> `timestamp jumps`; demurrage becomes `VISIBLE`, cross-checked against `buck_math` wasm predictions).
2. **Browser demo v1**: the interactive page – create agents (real registrations), mint `BuckCredit`, identity-gated `BUCK` transfers, time controls, live balances with demurrage – the whole Stage-1 world compiled for the browser (tevm is browser-capable by design; wasm kernels already web-target). Met by `core/js/demo/buckworld.html` (`make nix-core-demo-buckworld`): a DOM-free controller (`demo/src/app.js`) drives the story and is gated twice – once with node-loaded deps (`app.tevm.test.js`), and once as the EXACT shipped esbuild bundle executed headless under a minimal DOM stub (`demo.bundle.test.js`), which must complete the scripted opening (two kernel-ceremony citizens, credit line, bilateral handshake, payment, 30 simulated days) and light the bit-exact demurrage badge. Browser shims: npm `buffer` polyfill + an empty `fs` (an `@tevm/node` persistence path the browser never takes); bundle ~4.3 MB + the two web-target wasm kernels.
3. **Background agents + prices**: whale pin + roundtrip traders (Phase 1 agents) driving the pools on a seeded price stream while the user interacts; play/pause/speed. Met by `openMarket()/=marketTick()` in the demo controller (a real `TOKEN/USDC` pool beside the `BUCK` stack; `src/prices.js` seeded `BigInt` walk) + the page's market panel and tick/play/pause/speed controls; gated by `market.tevm.test.js` (pool tracks the walk to ≤ 50 bp with the trader churning, EXACT

pin with the whale alone, every snap and round-trip journaled by tag, demurrage bit-exact at the ACTUAL elapsed time) and the extended bundle gate.

4. **Journal parity** (the acceptance gate): a canonical mini-scenario defined once (`core/vectors/mini-scenario.json`: fixed targets – no cross-language RNG at all) and driven op-for-op, tag-for-tag by BOTH platforms – `alberta_buck.sim.mini` (python -> anvil) and `core/js/scenarios/mini.js` via `bin/mini-sim.mjs` (node -> tevm AND -> a fresh anvil). Met (`test_mini_parity.py`): python-anvil vs js-anvil journals are BYTE-EQUAL on every record – all semantic fields AND gas (26 records: 4 deploys, the one-pool build, 3 days of whale pins + round trips incl. the declared-revert SPL demo). js-tevm vs js-anvil: semantics exactly; tevm’s ethereumjs EVM prices a 4/26 minority differently (the callback-bearing SimLP liquidity mint + whale swaps, +~3.9k gas each) – summarized and bounded by the gate; anvil remains the fidelity anchor. Two session fixes landed en route: the Python `deploy()` gained the instance-label `tag`- seam the JS session already had, and `_revert_reason` now journals the BARE Solidity reason ("SPL"), the schema’s documented form – the pinned journal-sample fixture had always specified it; the implementation finally conforms.
5. **Instrument + charts**: a per-tick recorder in the demo controller (pool spots vs references, citizen balances/fees, K and basket value) and in-page SVG charts; the market panel graduates from numbers to time series.
6. **The equilibrium world in JS**: `buildEquilibriumWorld` ports `deploy.py`’s basket world – `BuckBasketProRata` + `TOKEN/BUCK` pools + the floating `BUCK/USDC` pool, CSV reference feeds bundled for the browser, whales pinning `TOKEN/USDC` to the feeds, a `PidKeeper` driving the controller – on the `REAL` routing periphery (see the SimLP risk note: `router.tevm.test.js` already proves the router on Tevm). Gate: a mini-equilibrium scenario’s py/js journals agree (the Stage-4 transcription pattern).
7. **Economic agents**: the prototype agents (`core/js/prototypes/`) go live – basket investor, mortgage retiree, plus a saver – with their ROI / cost-comparison chart panels.

8. **The participating user:** swap panel (USDC <-> BUCK <-> TOKENs via the real router), basket deposit/redeem with a receipt/NFT viewer, pledge form (createCredit).
9. **Embedding:** single-file build of the demo for slide decks / web pages; docs.

4.5.2 The JS agent doctrine (banked 2026-07-04)

Audience: the JS platform is written for a senior integration developer at a credit union or payments processor – someone who wires account systems to card networks and will use our JS/Python/Rust SDKs to build back-office BUCK services: deposit and credit product account values, investment values and histories, statements, web interfaces. Their first contact is the example agents, so THE AGENTS ARE THE PRODUCT: SDK documentation that happens to run.

The agent contract (all of it): a plain object with `setup(world)` and `act(world, day, tick)`; every chain mutation goes through `session.send(contract, fn, args, {tag, account, expect})` – journaled, expectation-declared, refusals first-class; analytics accumulate in a plain `this.ledger` array the chart panels read. No framework, no base class, no hidden state.

The world builder carries the plumbing: scenario builders provide labeled op helpers (`onboard` / `identityApprove` / `createCredit` today; `buyBuck` / `sellBuck` / `basketDeposit` / `basketRedeem` / `pledge` / `refUsd` / `receiptValueUsd` with the equilibrium world) so agents never touch custody mechanics, identity binding, or router encoding. Plumbing decisions (SimLP vs router, proxy vs EOA) change the world builder, never the agents.

Acceptance criterion (developer experience): every shipped example agent fits on one screen and reads cold to that developer. Enforced mechanically by the prototype gate (`core/js/test/prototypes.test.js` caps file length) and substantively by review.

Prototypes (`core/js/prototypes/`, banked ahead of the world that runs them; Stage 7 wires them live):

- `basket-investor.js` – deposits a constituent TOKEN into BuckBasket (BUCK + receipt out), marks the receipt to market daily, redeems at a horizon; books USD cost/mark/redeem rows for the ROI chart (dollar-days accounting mirroring the Python DM agents).
- `mortgage-retiree.js` – retires an amortizing mortgage with BuckCredit: `pledge -> activate -> draw -> pay out` the bank; monthly income then buys discounted BUCK (`basketValueInBuck > 1`) to retire

the draw, with forced deleverage when the controller tightens K past the position. Books BOTH columns of the mortgage-vs-BuckCredit cost chart from one ledger (counterfactual amortizing interest vs premium + spreads + forced costs).

4.6 Phase 5: scale-up

10^3 - 10^4 agent populations (extensive-margin adoption experiments from EQUILIBRIUM.md live here); throughput work (tx batching, session-level caching); OPTIONAL pyrevm backend behind the identical-vector gate.

- *Accept*: target population at acceptable wall-clock on anvil (or pyrevm with vectors proven identical); equilibrium acceptance gate still passes.

4.7 Phase 6: partner SDKs + wallet productization

buck-wallet (fountain management, tx signing) packaged for JS/Python/native; registrar/insurer/credit-union integration SDKs (same crates, thin service shims); Holochain zones consuming **buck-identity** != **buck-wallet**.

A named deliverable (banked 2026-07-04): the **back-office BUCK service** – a Node process an integrator runs beside their account system, exposing the BUCK endpoints their products need: balances and statements, deposit/credit product values, investment values and histories, an identity fountain isolating pools of back-office holdings, and Notes flows that shelter idle BUCK from demurrage and hide monetary-flow footprints from third parties. Stress-tested by pointing its **ChainSession** at a Tevm-hosted sim, anvil, or a testnet. The Notes cryptography is already in the wasm kernel (commitments, nullifiers, A2/B1 bindings, replayed by the JS suite); this productizes its orchestration.

- *Accept*: an end-to-end partner flow (sign Identity -> mint BuckCredit -> BUCK account product) driven purely through the SDKs on a Tenderly or testnet **ChainSession**; the back-office service survives a stress scenario against a Tevm or anvil sim.

5 Risks and open questions

- **Tevm fidelity/performance**: ethereumjs-EVM is slower than revm and its anvil-compatible surface is partial. Mitigation: demos are small populations; the identical-journal gate (Phase 4) catches semantic drift; anvil remains the fidelity anchor.

- **Thin periphery vs real routing tools** (SimLP): the pools were always real (`UniswapV3Factory` $\neq$ `UniswapV3Pool`); SimLP only simulated the PERIPHERY (mint/swap callbacks). RESOLVED 2026-07-04 (`core/js/test/router.tevm.test.js`): the Python sim's Universal Router recipe – the vendored artifact wired with a dummy Permit2 (pre-fund route, `payerIsUser=false`), OUR pool init-code-hash (locally compiled bytecode hashes differently than mainnet's), real WETH9 – deploys on Tevm at 5,346,812 gas (under the ~15M block budget) and routes a real `V3_SWAP_EXACT_IN`; `core/js/src/router.js` mirrors `alberta_buck/sim/router.py`. Doctrine: agent and USER swaps go through the real router (the code path integrators copy, and the surface real JS AMM tooling can be pointed at); SimLP remains for simulation god-modes only – whale pinning and liquidity seeding. In BUCK worlds the router is identity-bound public+carrying (`deploy.py` shows the bind). Real Permit2 (its canonical CREATE2 deploy) is optional follow-up for signature-based approvals.
- **Two agent platforms drifting**: mitigated by shared kernels, shared scenario vocabulary, shared journal diffing – and by keeping agents deliberately simple (a decision of record).
- **Crypto port correctness**: mitigated by the executable Python reference, frozen fixtures, and porting transcript-by-transcript with vectors at every step. The Poseidon constants and rng-stream pinning (`vectors.py`) must be treated as immovable.
- **wasm-bindgen/PyO3 version churn**: pin toolchains in the flake; the `holo-mutualcredit` `wasm-bindgen` pin is a cautionary tale.
- **Open**: npm package naming/scope; whether `core/js` adopts TypeScript from Phase 1 (leaning yes, once `tevm/viem` land as dependencies); wheel distribution channel for partners; when to split `core/` into its own repository (not before partners need it).